

SWEN-610 Midterm Study Guide

Foundations of Software Engineering · Fall 2026

This guide tells you what kinds of questions to expect and lists the concepts you should know well. The definitions here are a **starting point**: they are deliberately short. For each term, make sure you can explain it in your own words, give an example, and recognize it in an unfamiliar scenario. Use the lecture slides, class exercises, and your DB/REST projects to go deeper.

The guide covers more than the exam will. Everything here is fair game for the exam.

1. What to Expect

The midterm is a **75-minute, closed-book, paper exam** worth 100 points. There are four questions. Suggested times are printed on the exam to help you pace yourself. General examples of the questions are below, but the actual questions may vary.

Q	What you will do	Points	Time
1	Write SQL queries against a schema you have not seen before. They range from a simple SELECT-FROM-WHERE to a multi-table query with aggregates, GROUP BY, and HAVING. Your SQL is written by hand and is not executed.	30	~25 min
2	Draw a model from a written description. It will be one of: a domain model, a state chart, or a sequence diagram.	25	~20 min
3	Define one or more of REST's constraints, explain why the constraint is useful, and analyze an API design.	15	~10 min
4	Given a table with sample data, identify its normal form, give an anomaly, decompose it to a higher normal form, and explain why your design reaches that form.	30	~20 min

General advice. Read each question fully before writing. “Explain why” questions earn most of their points from the **reasoning**, not the final answer. Specify your assumptions whenever you feel as though it is necessary. We provide boxes for assumptions for questions where it is likely a good idea to specify them.

2. SQL

You worked through SQL in DB1–DB3. The exam expects you to write correct queries **without** a database to test against, so practice on paper. The SQL you write on the exam should be close to what you would write in a real database environment– but it does not need to be perfect (you won't be able to run them to be sure they're perfect). We will be primarily looking for good logical structure and reasonable understanding of how SQL operators work. We won't penalize you for typos or minor syntax errors.

2.1. Core query structure

SELECT	Chooses which columns (or computed expressions) appear in the result.
FROM	Names the table(s) the rows come from.
WHERE	Filters individual rows before any grouping happens.
ORDER BY	Sorts the result. ASC (default) or DESC. You can sort by multiple columns.

DISTINCT	Removes duplicate rows from the result.
Alias (AS)	A short or descriptive name for a table (FROM member m) or a column (COUNT(*) AS total).
LIMIT / OFFSET	Return only some rows, e.g. the top 5. Useful for pagination.
Predicates	=, <>, <, >=, BETWEEN, IN (. . .), LIKE with % wildcards, AND, OR, NOT.
NULL	“Unknown / missing.” Compare with IS NULL / IS NOT NULL; = NULL never matches.

2.2. Joins

Primary key (PK)	A column (or set of columns) that uniquely identifies each row.
Foreign key (FK)	A column that refers to the primary key of another table. It is what you join on.
INNER JOIN	Returns only rows that have a match in both tables. JOIN alone means inner join.
LEFT [OUTER] JOIN	Returns all rows from the left table, plus matching rows from the right; unmatched right columns are NULL. Use it when you need “including those with none.”
RIGHT / FULL OUTER JOIN	Mirror of left join / keep unmatched rows from both sides.
Join condition	The ON clause, usually fk = pk. Forgetting it produces every combination of rows (a cross product).

2.3. Aggregation

Aggregate function	Collapses many rows into one value per group.
COUNT(*)	Number of rows in the group, including rows with NULLs.
COUNT(col)	Number of rows where col is not NULL.
COUNT(DISTINCT col)	Number of different non-NULL values.
SUM(col)	Total of the values.
AVG(col)	Mean of the non-NULL values.
MIN(col) / MAX(col)	Smallest / largest value; works on numbers, dates, and text.
GROUP BY	Splits rows into groups that share the listed column values; aggregates are computed per group. Every non-aggregated column in SELECT must be in GROUP BY.
HAVING	Filters groups after aggregation, e.g. HAVING COUNT(*) >= 3.
NULLs & aggregates	All aggregates except COUNT(*) ignore NULLs.

2.4. WHERE vs. HAVING (the most common mistake)

WHERE decides which **rows** go into the groups. HAVING decides which **groups** come out. An aggregate like AVG(. . .) cannot appear in WHERE, because the groups don’t exist yet when WHERE runs.

Knowing the **logical** order in which a query is evaluated makes this obvious:

```
FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
```

A worked example using the Library domain from your project (your schema may differ):

```
-- Libraries that lent out at least 10 books in 2026,
-- with how many distinct users borrowed there.
```

```

SELECT l.name, COUNT(*) AS loans, COUNT(DISTINCT c.user_id) AS borrowers
FROM library l
      JOIN checkout c ON c.library_id = l.library_id
WHERE c.checkout_date >= '2026-01-01'      -- row filter
GROUP BY l.library_id, l.name
HAVING COUNT(*) >= 10                      -- group filter
ORDER BY loans DESC;

```

ON THE EXAM

Practice writing queries in increasing difficulty: single-table filter → one join → join + GROUP BY → multiple joins + WHERE + GROUP BY + HAVING. For each, say out loud what **one row** of the result represents before you write it.

3. Modeling

Models are simplified representations that help a team understand a problem or design before (and while) building it. Different models answer different questions.

3.1. Domain models

Domain model	A visual model of the real-world concepts in a problem domain and how they relate. It describes the problem, not the software.
Conceptual class	A thing, idea, or event in the domain (e.g., Book, Patron, Checkout). Usually a noun in the requirements.
Attribute	A simple data value that describes a conceptual class (e.g., a Book's title). If a "value" has its own attributes or identity, it is probably a class instead.
Association	A meaningful relationship between two conceptual classes, labeled with a verb phrase (e.g., Patron borrow s Book).
Multiplicity	How many instances of one class can be associated with one instance of the other: 1, 0..1, 1..*, 0..* (or *). Written on both ends of an association.
Description class	A class that holds shared information about a kind of item (e.g., BookTitle) as opposed to individual items (BookCopy).
Assumption	A decision you make where the requirements are silent or ambiguous. Always write it down.

A domain model is not a database schema or class diagram. It has no methods, no foreign keys, and no ID columns used as links; associations show the relationships instead.

Finding classes: scan the description for nouns and noun phrases, then ask which ones are important concepts the business keeps track of. Drop synonyms, things that are only attributes, and things outside the system.

3.2. State charts (state machine diagrams)

A state chart models how **one object** (or system) changes over time in response to events.

State	A condition the object is in for some period of time (e.g., Available, CheckedOut, Overdue).
Initial state	Filled circle; where the object starts.
Final state	Circle with a ring; the object's lifecycle ends (e.g., a book is withdrawn).

Transition	An arrow from one state to another.
Event (trigger)	Something that happens that may cause a transition (e.g., return).
Guard	A condition in square brackets that must be true for the transition to fire, e.g. [daysLate > 0].
Action / effect	Behavior that happens during a transition, written after a slash, e.g. return / computeFee.
Transition label syntax	event [guard] / action; each part is optional.

3.3. Sequence diagrams

A sequence diagram shows how **several participants** interact over time to carry out one scenario (often one use case).

Participant / lifeline	An actor or object, drawn as a box with a dashed vertical line beneath it. Time runs downward.
Message	An arrow from one lifeline to another, labeled with the call or request (e.g., GET / books/42).
Synchronous message	Solid line, filled arrowhead; the sender waits for a reply.
Asynchronous message	Solid line, open arrowhead; the sender does not wait.
Return message	Dashed arrow back to the caller with the result.
Activation bar	Thin rectangle on a lifeline showing when that participant is busy handling a message.
Combined fragment	A labeled box for control flow: alt (if/else), opt (if), loop (repetition).

ON THE EXAM

A state chart answers “what states can **this one thing** be in?” A sequence diagram answers “who talks to whom, in what order, for **this one scenario**?” A domain model answers “what concepts exist and how are they related?” Be able to say which model you would choose for a given question.

4. REST

REST (Representational State Transfer) is an architectural style for networked systems, described by Roy Fielding. An API is “RESTful” when it follows REST’s constraints.

4.1. Core constraints

Client-server	Separate the user interface (client) from data storage and processing (server) so each can evolve independently.
Stateless	Each request from the client contains all the information needed to understand it. The server stores no client session state between requests; that state lives on the client and is sent with each request (e.g., a token, a page number).
Cacheable	Responses say whether they can be cached and for how long, so clients and intermediaries can reuse them and avoid repeat requests.

Uniform interface	All resources are accessed the same general way. It has four parts (below).
Layered system	A client can't tell whether it is talking to the real server or an intermediary (load balancer, cache, gateway). Layers can be added without changing clients.
Code on demand (optional)	The server may send executable code (e.g., JavaScript) to extend the client.

The four parts of the uniform interface:

Identification of resources	Every resource has a unique identifier, a URI (e.g., /books/42).
Manipulation through representations	Clients work with representations of a resource (e.g., JSON) and send representations back to change it.
Self-descriptive messages	Each message carries enough information to process it: method, headers, media type (Content-Type), status code.
HATEOAS	Hypermedia As The Engine Of Application State: responses include links to related resources and available next actions.

4.2. Why statelessness matters

Be able to explain each of these **and why statelessness causes it**:

- **Scalability**: any server can handle any request, so you can add servers behind a load balancer.
- **Reliability**: a crashed server loses no session; the client just retries elsewhere.
- **Visibility / simplicity**: each request can be understood, logged, and tested on its own.
- **Cacheability**: self-contained requests make responses easier to cache.

The trade-off: requests may be larger, since context is resent each time.

Note that statelessness is about **session** state. The server still stores **resource** state (your database).

4.3. HTTP vocabulary you should know

Resource	Any named thing the API exposes (a book, a user, a collection of checkouts). Use nouns in URIs: /users/7/checkouts, not /getCheckouts.
GET	Retrieve a representation. Safe and idempotent.
POST	Create a new resource in a collection (or trigger processing). Neither safe nor idempotent.
PUT	Replace a resource entirely. Idempotent.
PATCH	Partially update a resource.
DELETE	Remove a resource. Idempotent.
Safe	The request does not change server state.
Idempotent	Making the same request once or many times has the same effect. (Same idea as idempotent unit tests in DB0!)
Status codes	200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 500 Server Error.
Query parameters	Options on a request that filter, sort, or paginate: /books?author=Shelley&page=2.

ON THE EXAM

Be ready to look at an API design and say whether it violates statelessness (does a request depend on something the server “remembers” from an earlier request?) and how to fix it.

5. Database Normalization

Normalization organizes tables to reduce redundancy and prevent anomalies. Each normal form builds on the one before it.

5.1. Vocabulary

Functional dependency (FD)	$A \rightarrow B$ means: if you know the value of A, there is exactly one value of B. (Each student ID determines one name.)
Candidate key	A minimal set of columns that uniquely identifies each row. The primary key is the one chosen.
Prime / non-key attribute	A column that is part of some candidate key / one that isn't.
Partial dependency	A non-key attribute depends on only part of a composite key.
Transitive dependency	A non-key attribute depends on another non-key attribute, which depends on the key: $\text{key} \rightarrow X \rightarrow Y$.
Redundancy	The same fact stored in more than one row.
Update anomaly	Changing one fact requires changing many rows; miss one and the data contradicts itself.
Insertion anomaly	You can't record one fact without also recording an unrelated one (e.g., can't add a new course until a student enrolls).
Deletion anomaly	Deleting one fact accidentally deletes another (e.g., removing the last enrollment loses the course).
Decomposition	Splitting a table into smaller tables connected by foreign keys.
Lossless join	A decomposition is lossless if joining the new tables reproduces exactly the original data.

5.2. The normal forms

1NF	Every column holds a single, atomic value (no lists, no repeating groups like phone1, phone2), and each row is uniquely identifiable by a key.
2NF	In 1NF, and no partial dependencies: every non-key attribute depends on the whole key. (A table with a single-column key that is in 1NF is automatically in 2NF.)
3NF	In 2NF, and no transitive dependencies: non-key attributes depend only on the key, not on other non-key attributes.

A classic memory aid for 3NF: every non-key attribute depends on “**the key** (1NF), **the whole key** (2NF), and **nothing but the key** (3NF).”

5.3. A process for normalization questions

1. Identify the primary key. Is it composite?
2. List the functional dependencies the business rules imply.

3. Check 1NF: atomic values, a key exists.
4. Check 2NF: does any non-key attribute depend on only part of the key?
5. Check 3NF: does any non-key attribute depend on another non-key attribute?
6. Decompose: give each determinant its own table with the attributes it determines, and leave behind a foreign key.
7. Explain the result by naming the dependency each new table removes.

BEYOND THE EXAM (still part of the course)

Denormalization is sometimes chosen on purpose (e.g., for read performance or reporting), accepting redundancy in exchange for fewer joins. Be able to discuss the trade-off.

ON THE EXAM

Labels alone earn few points. Saying “it’s in 1NF” is worth less than saying **which** dependency keeps it from being in 2NF. When you decompose, mark every PK and FK, and tie your explanation to the specific dependencies you removed.

6. How to Study

- **Redo your DB project queries on paper**, without looking at your code. Then check them against your repository.
- **Write your own questions.** For each SQL clause, aggregate, and normal form above, invent a small example.
- **Explain it to someone.** For each term in this guide, explain it aloud in two sentences and give one example. If you can’t, that’s what to study.
- **Time yourself.** Try one domain model in 20 minutes and one normalization problem in 20 minutes.
- **Bring questions** to office hours before the exam.
- **Use Agent Support** Ask an agent to generate questions for practice, explanations, or clarifications on topics you find challenging.